Ingegneria del Software

SunSpot: Gestionale per la prenotazione digitale di stabilimenti balneari

Leonardo Soldani - 7139732

Marzo 2026

Indice

1 Introduzione **1**

2 Ambiente e tecnologie utilizzate **1**
2.1 Linguaggio e Tool di Build 1
2.2 Database e Persistenza 1
2.3 Librerie e Containerizzazione 1
2.4 Testing e Analisi 1

3 Use Case **1**

4 Database **2**
4.1 Modello ER 2
4.2 Dizionario dei dati 2
4.3 Tabelle delle Regole 4

1 Introduzione

Il progetto SunSpot nasce dall'esigenza di digitalizzare e modernizzare il settore degli stabilimenti balneari, un ambito che ancora oggi mostra una certa resistenza all'adozione di piattaforme web integrate. L'obiettivo principale è fornire un'infrastruttura capace di permettere all'utente di prenotare in modo rapido e intuitivo un ombrellone presso uno dei lidi aderenti al servizio, trasformando la pianificazione di una giornata al mare in un'operazione a portata di click.

Il sistema è stato progettato con l'idea di soddisfare le esigenze di tre tipologie di utenti:

- **Utente finale (Customer)**: a cui viene fornita la possibilità di prenotare uno o più ombrelloni o tende con la possibilità di personalizzare la propria esperienza richiedendo lettini, sedie, sdraio extra e cabine.
- **Gestore stabilimento (Owner)**: che dispone di innumerevoli funzionalità per la personalizzazione della propria spiaggia virtuale, partendo da una sezione per poter pubblicizzare i propri servizi arrivando alla possibilità di suddividere il proprio stabilimento in diverse aree di prezzo.
- **Admin (Gestore dell'applicazione)**: con lo scopo di regolare e controllare l'ambiente commerciale messo a disposizione di gestori e utenti finali. Si occupa dell'inserimento della spiaggia all'interno dell'ambiente e del rispettivo gestore, oltre ad applicare sanzioni (ban) qualora ce ne fosse necessità.

Per garantire un ecosistema sicuro e affidabile, SunSpot integra un sistema di regolazione basato su recensioni verificate e un meccanismo di segnalazione (report) tra utenti e gestori, volto a preservare l'integrità del servizio e a migliorare costantemente l'esperienza d'uso per tutti gli attori coinvolti.

2 Ambiente e tecnologie utilizzate

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie:

2.1 Linguaggio e Tool di Build

- **Java 25** - Scelto per l'utilizzo delle funzionalità più recenti della JDK.
- **Maven 3.8.5** - Strumento di build e gestione delle dipendenze.

2.2 Database e Persistenza

- **MySQL 8.3.0** - Database relazionale utilizzato per la persistenza dei dati.
- **MySQL Connector/J** - Driver ufficiale per la gestione della connessione al database.

2.3 Librerie e Containerizzazione

- **BCrypt 0.10.2** - Utilizzata per l'hashing e la gestione sicura delle password.
- **Docker & Docker Compose 3.8** - Utilizzati per containerizzare il database MySQL, garantendo un ambiente di esecuzione isolato, persistente e facilmente riproducibile.

2.4 Testing e Analisi

- **JUnit 5.12.1** - Framework per l'esecuzione dei test unitari.
- **Mockito 5.23.0** - Utilizzato per la simulazione di dipendenze durante i test (mocking).
- **JUnit Platform Suite** - Strumento per l'organizzazione e l'esecuzione dei test.

3 Use Case

I seguenti template descrivono alcuni dei principali casi d'uso identificati per l'applicazione durante la fase di analisi e progettazione. Ogni caso d'uso è descritto utilizzando una struttura standardizzata: ID, Nome, Attori, Descrizione, Precondizioni, Postcondizioni, Flusso principale, Flussi alternativi e Business Rules.

4 Database

Per la gestione della persistenza dei dati è stato utilizzato il DBMS **MySQL**, configurato all'interno di un ambiente di containerizzazione **Docker**. L'interazione tra l'applicazione Java e il database avviene tramite l'interfaccia **JDBC**, utilizzando il driver ufficiale **MySQL Connector/J**.

L'integrità del dato è assicurata a livello nativo attraverso una rigorosa definizione di vincoli (Foreign Key, Unique e Check) implementati nello script di creazione. Tale approccio permette di delegare al database la validazione delle informazioni critiche, mentre le verifiche legate alla **logica di business** sono gestite programmaticamente all'interno del codice Java dell'applicazione.

4.1 Modello ER

Il modello Entity-Relationship (fig. 1) omette la relazione diretta tra *Beaches* e *Bookings* per evitare ridondanze concettuali, essendo il legame già mediato dalle entità *Zones* e *Spots*. In fase di implementazione del database fisico, tuttavia, tale relazione è stata introdotta per ottimizzare le performance delle query di ricerca, riducendo la complessità delle operazioni di join.

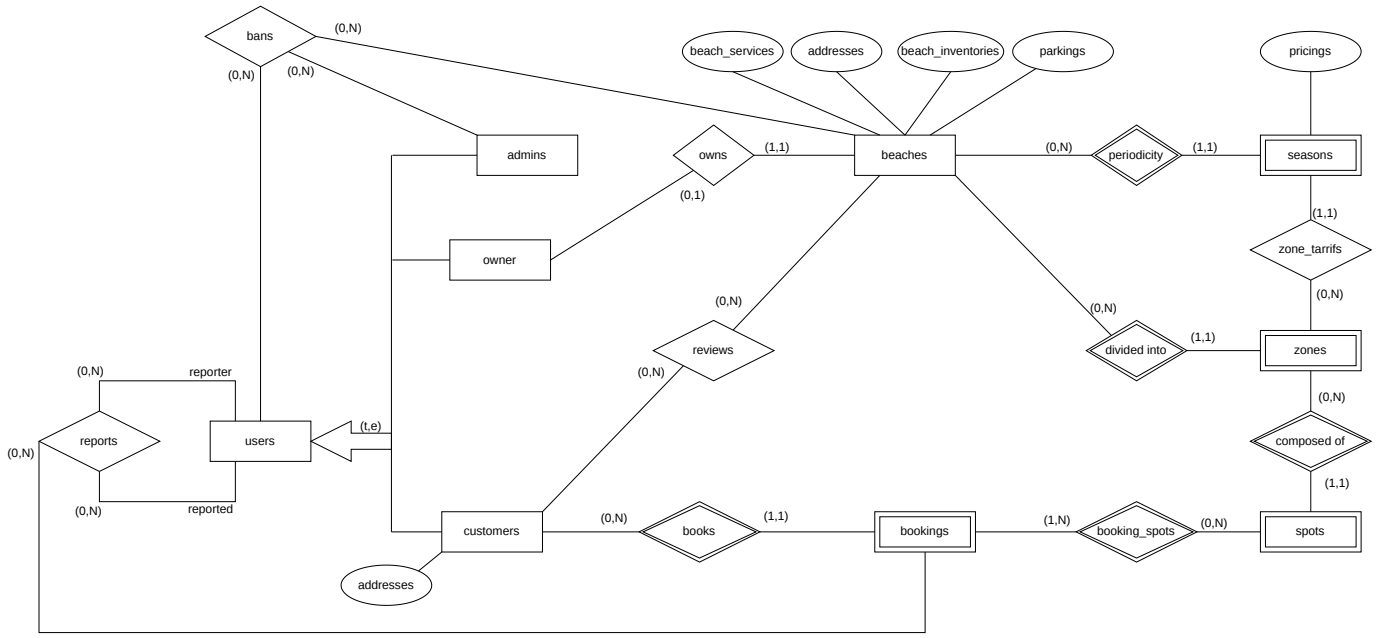

Figura 1: Modello Entity Relationship (Schema Concettuale)

4.2 Dizionario dei dati

Il dizionario (fig. 2 e 3) presenta per *Spots* e *Bookings* attributi identificativi esterni derivanti dalle relazioni con altre entità, che dovrebbero svolgere il ruolo di chiave primaria per le entità citate. Per semplificare le interrogazioni e la realizzazione del db si è optato per l'utilizzo di una chiave primaria artificiale (*id*), preservando la valenza identificativa di questi attributi tramite vincoli `UNIQUE` e `NOT NULL`, come dettagliato nella tabella dei vincoli (fig. 4).

Entità	Descrizione	Attributi	ID
Users	Generalizzazione degli utenti	name, surname, username, email, hashPassword	id
Admins	Amministratori di sistema	OTP	id (*Users*)
Owners	Gestori degli stabilimenti	active, OTP	id (*Users*)
Customers	Utenti (Clienti)	phoneNumber, active	id (*Users*)
Beaches	Stabilimenti balneari	name, description, phoneNumber, extraInfo, active, closed	id
Parkings	**Attributo composto** di Beaches. Info parcheggio	nAutoPark, nMotoPark, nElectricPark, CCTV	-
Beach Services	**Attributo composto** di Beaches. Servizi offerti	bathrooms, showers, pool, bar, restaurant, wifi	-
Beach Inventories	**Attributo composto** di Beaches. Inventario	countExtraSdraio, countExtraLettini, countExtraSedie, countCamerini	-
Addresses	**Attributo composto**. Indirizzi di utenti e spiagge	street, streetNumber, city, zipCode, country	-
Pricings	Listino prezzi base	priceLettino, priceSdraio, priceSedia, priceParking	-
Seasons	Periodi temporali (Stagionalità)	startDate, endDate, name	name, beachId (*Beaches*)
Zones	Aree geografiche della spiaggia	name	name, beachId (*Beaches*)
Spots	Postazioni prenotabili	type, row, column, zoneName (*Zones*), beachId (*Beaches*)	id
Bookings	Dati della prenotazione	date, extraSdraio, extraLettini, extraSedie, camerini, autoPark, motoPark, electricPark, totalPrice, status, callerName, callerPhone, customerId (*Customers*), beachId (*Beaches*)	id

Figura 2: Dizionario delle Entità

Relazioni	Descrizione	Entità coinvolte	Attributi
Bans	Gestione allontanamenti	Admins (0,N), Users (0,N), Beaches (0,N)	reason, banType, createdAt
Owns	Proprietà dello stabilimento	Owners (0,1), Beaches (1,1)	-
Reviews	Valutazioni degli utenti	Customers (0,N), Beaches (0,N)	rating, comment, createdAt
Books	Assegnazione prenotazione	Customers (0,N), Bookings (1,1)	-
Periodicity	Associazione stagionale	Beaches (0,N), Seasons (1,1)	-
Zone Tariffs	Prezzi per zona specifica	Seasons (1,1), Zones (0,N)	priceOmbrellone, priceTenda
Divided into	Partizione della spiaggia	Beaches (0,N), Zones (1,1)	-
Composed of	Layout dei posti	Zones (0,N), Spots (1,1)	-
Booking Spots	Occupazione fisica spot	Bookings (1,N), Spots (0,N)	date
Reports	Segnalazioni problemi	Users (0,N), Bookings (1,1)	status, description, createdAt

Figura 3: Dizionario delle Relazioni

4.3 Tabelle delle Regole

Di seguito vengono riportati i vincoli di integrità e le regole di derivazione che governano la logica della persistenza dei dati. In merito alla regola di RD1 (fig. 5), si è scelta la persistenza dell'attributo *totalPrice* nel database, al fine di ridurre il carico computazionale di consultazione dei dati.

Codice	Regola di Vincolo (RV)
RV1	L'email e lo username di un utente (*Users*) devono essere univoci a livello di sistema.
RV2	Il numero di telefono (*phoneNumber*) di un cliente deve essere univoco.
RV3	Ogni spiaggia (*Beaches*) deve riferirsi obbligatoriamente a un proprietario (*Owners*) e a un indirizzo (*Addresses*) esistenti in modo univoco.
RV4	Il numero di posti auto, moto ed elettrici (*Parkings*) deve essere maggiore o uguale a zero.
RV5	Le quantità dell'inventario (*beach_inventories*), come ombrelloni e lettini, devono essere maggiori o uguali a zero.
RV6	Tutti i prezzi definiti nei listini (*Pricings*) e nelle tariffe per zona (*zone_tariffs*) devono essere maggiori o uguali a zero.
RV7	La data di inizio di una stagione (*startDate*) deve essere strettamente precedente alla data di fine (*endDate*).
RV8	Non possono esistere due postazioni (*Spots*) con la stessa riga e colonna nella medesima zona e spiaggia.
RV9	Le coordinate di riga (*row*) e colonna (*column*) di uno *Spot* devono essere maggiori o uguali a zero.
RV10	Un cliente non può effettuare più di una prenotazione (*Bookings*) per la stessa spiaggia nella medesima data.
RV11	Uno *Spot* non deve essere associato a più di una prenotazione (*booking_spots*) nella stessa data.
RV12	Le quantità di servizi extra richiesti in una prenotazione (lettini, sdraio, ecc.) devono essere maggiori o uguali a zero.
RV13	Un utente non può ricevere più di un ban attivo (*Bans*) per la stessa spiaggia o per l'intera applicazione.
RV14	Il rating di una recensione (*Reviews*) deve essere un valore intero compreso tra 1 e 5.
RV15	Un cliente può inserire al massimo una recensione per ogni spiaggia.
RV16	Ogni utente può emettere un solo report (*Reports*) per una specifica prenotazione.
RV17	Utilizzo identificatore surrogato (id) per Spots, deve esistere vincolo di unicità tra: *row*, *column*, *zoneName* e *beachId*.
RV18	Utilizzo identificatore surrogato (id) per Bookings, deve esistere vincolo di unicità tra: *customerId*, *beachId* e *date*.
RV19	La spiaggia associata a una prenotazione (*beachId* in *Bookings*) deve corrispondere alla spiaggia a cui appartengono i singoli posti prenotati (*beachId* in *Spots*).

Figura 4: Tabella dei vincoli di integrità (RV)

Codice	Regola di Derivazione (RD)
RD1	Il prezzo totale (*totalPrice*) di una prenotazione si ottiene sommando la tariffa dello spot (basata su zona e stagione) al costo dei servizi extra e dei parcheggi prenotati (quantità moltiplicata per il rispettivo prezzo unitario).

Figura 5: Tabella delle regole di derivazione (RD)